Distributed Audio Processing Pipeline with Observability

Final Report for the Distributed Systems Course AY 2025/2026

Matteo Cardellini
matteo.cardellini@studio.unibo.it

April 7, 2026

This work presents a distributed, event-driven pipeline for asynchronous audio processing, designed as a practical case study for distributed systems principles and operational observability.

Users submit audio files through a FastAPI service. The API stores binary objects in MinIO, persists job metadata in PostgreSQL, and publishes processing requests to RabbitMQ. A decoupled worker consumes queued jobs, extracts core audio metadata (duration, sample rate, channels), and writes results back to the database. The same API exposes upload, job tracking, and result retrieval endpoints, enabling a complete end-to-end workflow.

The architecture separates ingestion and processing through asynchronous messaging, so API and worker instances can scale independently. Reliability is modeled through durable queue-based communication and explicit job-state transitions (PENDING, PROCESSING, DONE, FAILED), while observability is supported via Prometheus-compatible metrics for throughput, latency, queue backlog, and worker activity.

The stack is containerized with Docker and structured for Kubernetes-based deployment and scaling experiments. The project shows how message-driven coordination, persistent state management, and monitoring can be combined into a reproducible distributed audio-processing system.

Disclaimer (if needed)

During the preparation of this work, the author used GitHub Copilot for language refinement and drafting support. All technical choices, validation activities, and final content were reviewed and approved by the author, who takes full responsibility for the report.

1 Concept

Type of Product

The developed product is a **distributed, service-oriented application** for asynchronous audio processing. From an architectural perspective, it is composed of **three cooperating software components**:

- a **web API** (FastAPI) that exposes upload, job-status, and result-retrieval endpoints;
- a **background worker service** that consumes queued jobs and executes audio processing tasks;
- a **lightweight browser-based frontend** used as an interactive client for manual usage and demo scenarios.

Therefore, the project is primarily a distributed web-service system, complemented by a simple web application for user interaction, rather than a standalone library or CLI-only tool.

Use Case Description

The system supports **asynchronous audio-processing requests** submitted by users through an HTTP API. In a typical scenario, a user uploads an audio file, receives a **job identifier**, and later queries the job state and extracted results. This interaction model avoids long synchronous waits and is suitable for workloads where processing time is variable.

Users access the platform remotely through a web browser or an API client, typically from personal computers and, when needed, from mobile devices with HTTP-capable tools. Interaction frequency is naturally bursty: short upload sessions can generate multiple jobs in sequence, followed by periodic status checks until processing completes.

The platform stores both **binary** and **structured** data. Audio files are persisted in object storage (MinIO), while job metadata and processing outcomes are persisted in PostgreSQL. Stored information includes job identifiers, file references, processing states, timestamps, and extracted audio metadata such as duration, sample rate, and number of channels.

At the architectural level, the system involves distinct roles: client users submitting requests, the API service coordinating ingestion and orchestration, and worker services executing background processing. This role separation is a core part of the distributed use case and enables clearer responsibility boundaries across components.

Why Distribution Is Needed

Distribution is a **structural requirement** of this project, not only an implementation preference. Audio processing tasks can take variable and non-trivial execution time,

so coupling upload handling and processing in a single synchronous component would reduce responsiveness and limit throughput. By separating ingestion (API) from execution (workers) through a message broker, the system can accept requests quickly and process them asynchronously.

This design enables **independent horizontal scaling** of components: API instances can scale to absorb request bursts, while worker replicas can scale according to queue backlog and processing demand. It also promotes resource sharing, since multiple workers consume from the same queue and operate on a common persistent state stored in PostgreSQL and MinIO.

Distribution also improves **fault isolation** and operational resilience. If a worker instance fails during processing, new requests can still be accepted by the API, and queued jobs remain available for other workers. Combined with explicit job-state transitions and persistent storage, this model provides a robust foundation for reliability, observability, and incremental system evolution.

2 Requirements Elicitation and Analysis

This section defines **what** the system must provide from a functional and quality perspective. Requirements are identified with unique IDs and paired with explicit acceptance criteria used later in validation.

Glossary

- **Job**: one asynchronous processing request created after an audio upload.
- **Job state**: lifecycle value associated with a job (PENDING, PROCESSING, DONE, FAILED).
- **Queue backlog**: number of jobs waiting to be processed by workers.
- **Processing result**: extracted audio metadata returned by the worker and persisted by the platform.

Functional Requirements

- **FR1: Audio upload**. The system shall allow a user to submit an audio file and create a processing job. **Acceptance criterion:** after a valid upload request, the API returns success with a new job identifier and an initial job state.
- **FR2: Asynchronous execution**. The system shall execute audio processing asynchronously with respect to request submission. **Acceptance criterion:** upload requests complete without waiting for final processing, while the corresponding job continues in background.

- **FR3: Job status retrieval**. The system shall expose an endpoint to retrieve the current state of a job by identifier. **Acceptance criterion:** querying an existing job returns one valid state among PENDING, PROCESSING, DONE, FAILED.
- **FR4: Result retrieval**. The system shall expose an endpoint to retrieve processing results when available. **Acceptance criterion:** for jobs in DONE state, the API returns persisted extracted metadata; for unfinished jobs, it returns a consistent status response.
- **FR5: Persistent storage**. The system shall persist uploaded audio objects and job metadata. **Acceptance criterion:** after restart of application containers, previously created jobs and stored files remain available.
- **FR6: Failure visibility**. The system shall make processing failures observable through job state and error information. **Acceptance criterion:** if processing fails, the job is marked FAILED and failure details are retrievable through API/database inspection.

Non-Functional Requirements

- **NFR1: Scalability**. The system should support horizontal scaling of API and worker components. **Acceptance criterion:** increasing worker replicas increases processing throughput under comparable load.
- **NFR2: Responsiveness**. Upload operations should remain fast even when processing is slow. **Acceptance criterion:** upload endpoint remains responsive because processing is decoupled and queued.
- **NFR3: Reliability**. Jobs should not be lost during normal component restarts or transient worker issues. **Acceptance criterion:** queued or persisted jobs are recoverable and continue through lifecycle after restart.
- **NFR4: Observability**. The platform should expose measurable operational signals. **Acceptance criterion:** Prometheus-compatible metrics for requests, processing activity, and failures are exposed by services.
- **NFR5: Maintainability**. The architecture should keep service responsibilities clearly separated. **Acceptance criterion:** API and worker can be developed, deployed, and scaled independently without changing external contract.

Implementation Constraints

- **IR1: Service implementation language**. Core services shall be implemented in Python. **Acceptance criterion:** API and worker codebases are Python projects with reproducible dependency management.

- **IR2: Messaging middleware**. Asynchronous coordination shall use RabbitMQ. **Acceptance criterion:** job dispatch and consumption happen through RabbitMQ queues/exchanges.
- **IR3: Data technologies**. Job metadata shall be persisted in PostgreSQL and audio objects in MinIO. **Acceptance criterion:** no alternative primary store is required for the main pipeline flow.
- **IR4: Containerized deployment**. Services shall be runnable via container orchestration for reproducibility. **Acceptance criterion:** the full stack can be started with Docker Compose and deployed to Kubernetes manifests/charts provided in the repository.

2.1 Relevant Distributed System Features

For this project, distributed-system features have different priorities.

- **Scalability (high relevance).** The pipeline must absorb burst uploads and variable processing demand. Independent scaling of API and workers is therefore essential.
- **Fault tolerance and dependability (high relevance).** Worker or network issues must not cause immediate job loss. Durable messaging, persistent state, and explicit job transitions are central to reliability.
- **Resource sharing (high relevance).** Multiple workers share queue workload and common storage backends. Coordination around shared queue and shared state is intrinsic to the architecture.
- **Performance and concurrency (high relevance).** Many jobs may execute concurrently, and throughput under load is a key quality objective.
- **Evolvability and maintainability (high relevance).** Service separation enables incremental changes (API, worker, observability) with limited cross-impact.
- **Security and trust (medium relevance).** The system includes authentication and controlled API access, but it is not designed for highly sensitive regulated workloads.
- **Transparency (medium relevance).** Users are abstracted from internal distribution details, but operational distribution remains explicit for developers and operators.
- **Openness and interoperability (medium relevance).** The system relies on widely adopted protocols and components (HTTP, AMQP, SQL), which simplifies integration and portability.

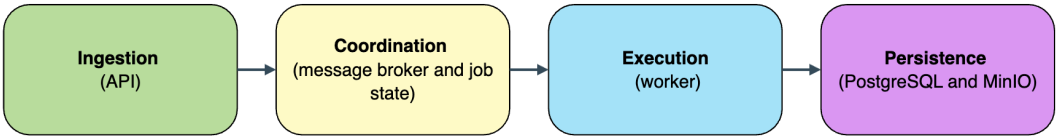

Figure 1: Four-block pipeline: ingestion, coordination, execution, and persistence.

- **Economy and costs (medium relevance).** Containerization and local orchestration reduce setup and experimentation costs while preserving realistic deployment patterns.

3 Design

This section describes how the system design addresses the requirements in section 2, with a focus on scalability, fault isolation, resource sharing, and observability. The architecture is message-driven: ingestion, background processing, and persistence are separated into dedicated components so each part can evolve and scale independently. This separation keeps API requests responsive under load, while worker capacity can increase as queue backlog grows. Components coordinate asynchronously through explicit job lifecycle states persisted in shared storage.

The next subsections present the design in order: architectural style, infrastructure, modelling, interaction, behaviour, data consistency, fault tolerance, availability, and security.

3.1 Architecture

The system follows an **event-driven, service-oriented architecture**. It combines two interaction styles:

- **synchronous request-response** for user-facing API operations (upload, status, result retrieval);
- **asynchronous message-driven processing** for background job execution through RabbitMQ.

This architectural style was chosen to satisfy the most important requirements. First, it keeps the API responsive because request handling is separated from long-running processing tasks. Second, it enables independent scaling of API and worker components, which directly supports throughput growth under variable load. Third, it improves fault isolation: temporary worker issues do not block request intake, and queued jobs remain processable by other workers.

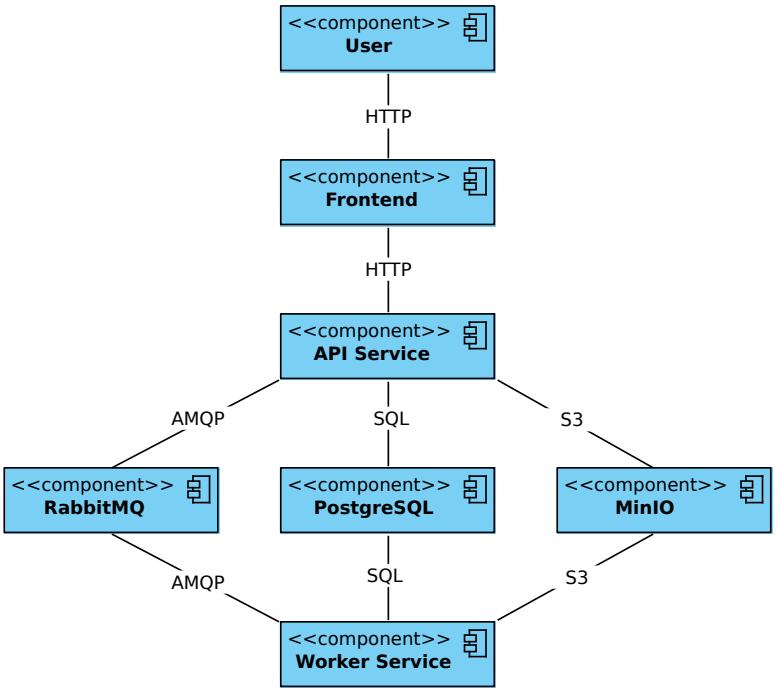

Figure 2: Runtime components and their communication protocols.

In practical terms, the architecture can be seen as a small pipeline with clear stages: **ingestion** (API), **coordination** (message broker and job state), **execution** (worker), and **persistence** (PostgreSQL and MinIO). This separation also improves maintainability, since each stage can evolve with limited impact on the others. A visual summary of this **four-block pipeline** is shown in fig. 1.

3.2 Infrastructure

The infrastructure is built around six runtime components. Figure 2 provides a simplified overview of their roles and primary communication protocols (HTTP, AMQP, SQL, S3).

- **Frontend client**: a lightweight web interface for manual interaction;
- **API service**: handles HTTP requests, upload orchestration, and status/result queries;
- **Worker service**: consumes queued jobs and executes audio processing;
- **RabbitMQ**: message broker used for asynchronous coordination between API and workers;
- **PostgreSQL**: persistent relational store for job metadata and processing outcomes;

- **MinIO**: object storage for uploaded audio files.

Kubernetes is the default deployment model: each component runs as one or more pods, while Services provide stable endpoints for internal and external traffic. This setup enables horizontal scaling of stateless parts, especially workers, without changing application logic. Service discovery is DNS-based through Kubernetes Service names, which removes hard-coded addresses and improves portability. Traffic distribution is handled by platform primitives: API HTTP requests are balanced at Service level, and processing load is distributed as workers compete for RabbitMQ messages. An external load balancer remains optional, depending on the target cluster environment.

3.3 Modelling

Figure 3 captures the core domain model of the asynchronous pipeline. The primary **domain entities** are `User`, `AudioJob`, and `ProcessedAudio`, with `AudioFeatures` as a value object associated with processing results. In this model, each uploaded file creates exactly one `AudioJob`, identified by a unique ID, owned by a user, and tracked through the lifecycle states `PENDING`, `PROCESSING`, `DONE`, and `FAILED`. When processing completes successfully, `ProcessedAudio` stores extracted metadata such as duration, sample rate, and channel count.

Entity-to-infrastructure mapping follows component responsibilities. Identity and job metadata are persisted in PostgreSQL, binary audio content is persisted in MinIO, and execution coordination is delegated to RabbitMQ. At runtime, API-side services create jobs, validate access, and expose query operations, while worker-side services consume queued jobs, execute processing logic, and apply lifecycle transitions to persistent state.

The **domain events** considered by the model are: *job created*, *job queued*, *processing started*, *processing completed*, and *processing failed*. These are reflected in three **message categories** exchanged among components: *commands* (e.g., enqueue processing), *events* (state transitions), and *queries* (status and result retrieval).

The **overall system state** includes user identity and role data, job ownership and lifecycle metadata, object-storage references, extracted audio features, timestamps, and error diagnostics. This state is intentionally externalized in durable stores so that execution remains recoverable after restarts and observable across API and worker components.

3.4 Interaction

Component interaction is structured around four recurrent runtime flows that cover the complete user-facing lifecycle: authentication, upload and job creation, background processing, and status/result retrieval. Within these flows, synchronous HTTP is used for client-facing request-response operations, whereas asynchronous AMQP messaging decouples ingestion from execution between API and workers. This hybrid interaction model provides immediate feedback to clients while allowing processing to complete in the background.

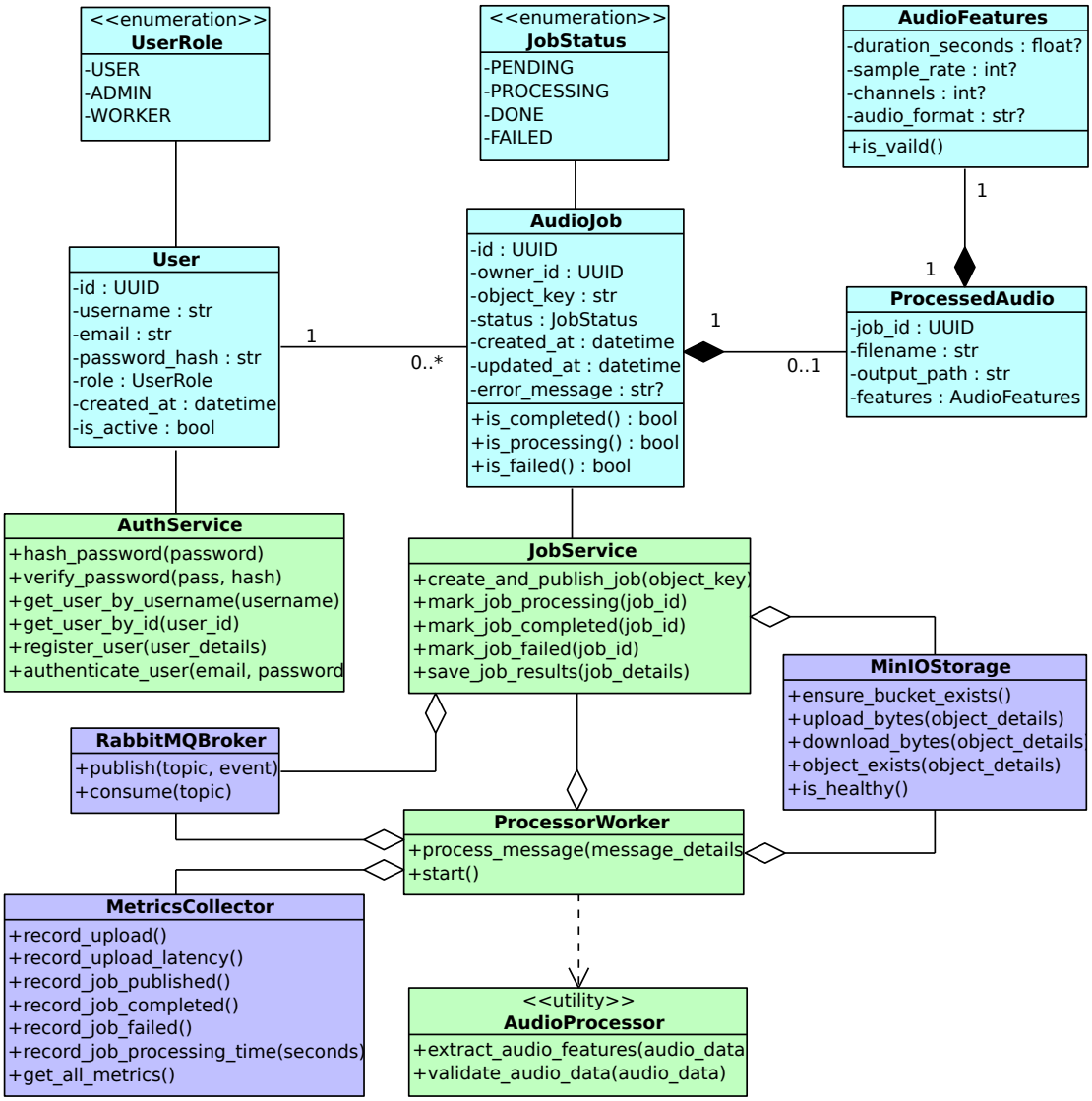

Figure 3: Class diagram showing the main domain entities, their relationships, and how they map to infrastructural components.

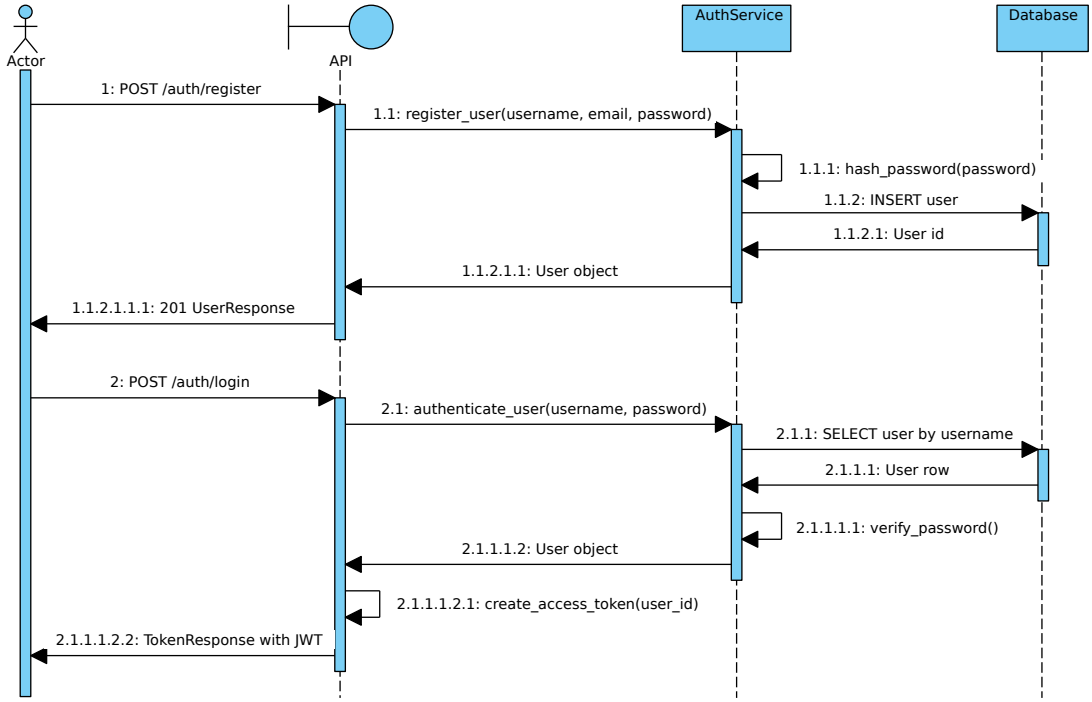

Figure 4: Sequence for user registration and login, leading to JWT issuance and authenticated API usage.

Flow 1: Authentication (register and login)

As shown in fig. 4, a new user first registers through the API. The service validates the payload, persists the user in PostgreSQL, and returns a successful registration response. For subsequent access, the login request triggers credential verification followed by JWT issuance. The client then includes the token in the `Authorization` header when invoking protected endpoints.

Flow 2: Upload and job creation

In fig. 5, an authenticated client uploads an audio file to the API. The API stores the binary object in MinIO, creates the corresponding `AudioJob` in PostgreSQL with initial state `PENDING`, and publishes a processing command to RabbitMQ. The client receives the job identifier immediately; therefore, request latency is independent of processing time. This is the primary ingestion path and the main integration point between synchronous and asynchronous communication.

Flow 3: Worker processing

Figure 6 details the execution path after a message becomes available in RabbitMQ. The worker consumes the job message, updates the job state to `PROCESSING`, fetches the

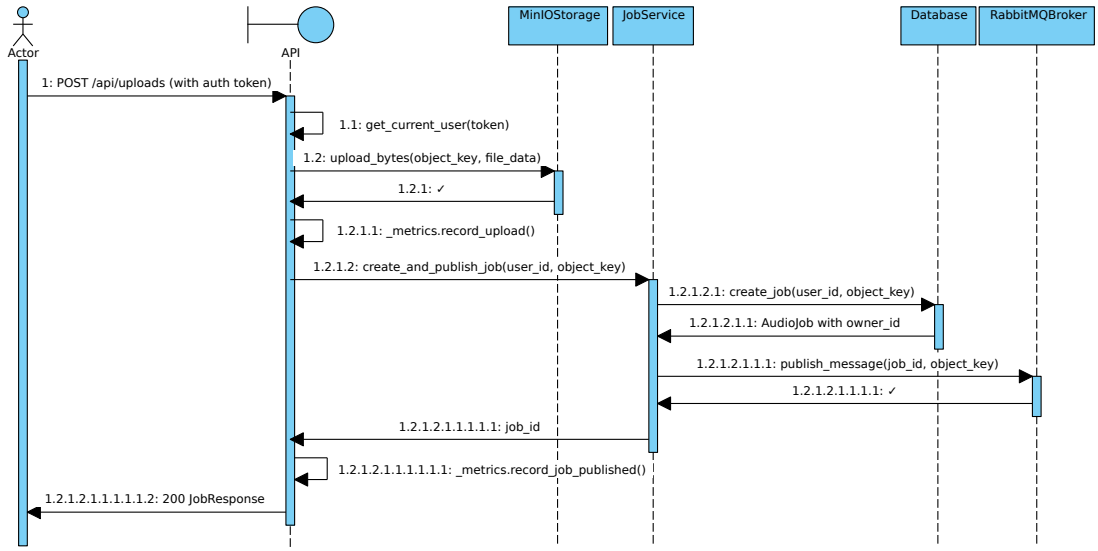

Figure 5: Sequence for file upload, persistent job creation, and asynchronous dispatch to the processing queue.

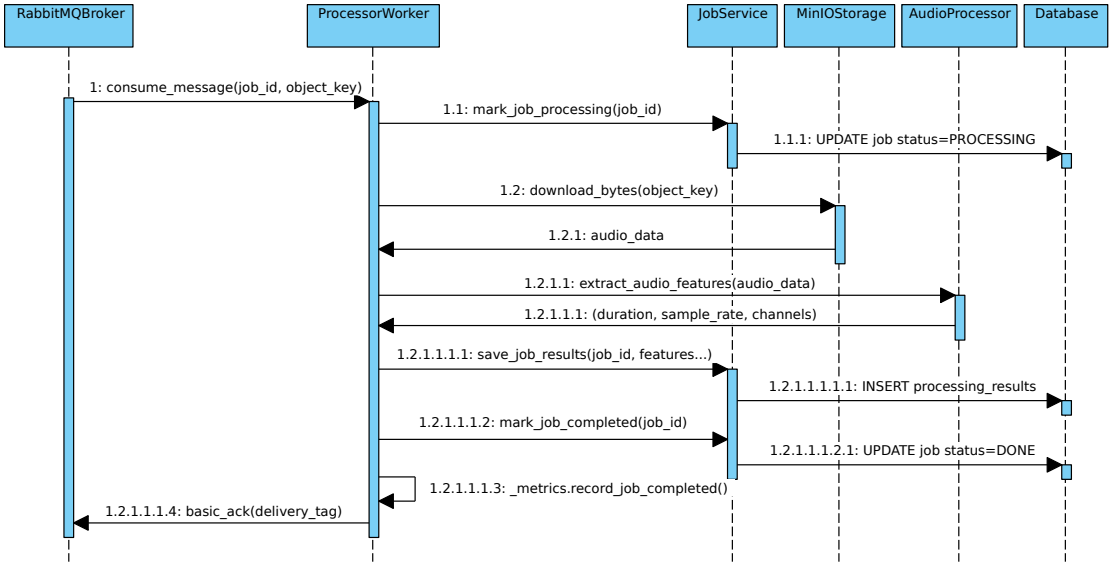

Figure 6: Sequence for queue consumption, processing execution, and final state transition update.

source object from MinIO, extracts audio metadata, and persists outputs in PostgreSQL. On success, the state transitions to `DONE`; on failure, it transitions to `FAILED` with diagnostic details. Broker acknowledgement is sent only after the processing attempt is finalized, supporting robust at-least-once consumption semantics.

Flow 4: Job status and result retrieval

Finally, fig. 7 describes the read path used after submission. The client periodically queries job status through the API, which retrieves the current lifecycle value from PostgreSQL. While processing is ongoing, the API returns consistent intermediate responses; once the state is `DONE`, the result endpoint returns persisted metadata (duration, sample rate, channels).

Taken together, these four flows implement a coherent interaction pattern: state-changing ingestion operations remain short and synchronous, compute-intensive work is decoupled and asynchronous, and completion is exposed through explicit state queries.

3.5 Behaviour

Behaviour in this system is centered on two complementary state machines. The first one describes the **business lifecycle** of an `AudioJob`, while the second one describes the **runtime behavior** of the worker while it consumes and processes queue messages. Together, they clarify which component updates state, which state is persistent, and which state is only internal to execution.

Figure 8 shows the persistent job lifecycle stored in PostgreSQL. When the API receives a valid upload, it creates the job in `PENDING` state. The worker then advances the job to `PROCESSING` when it starts consuming the corresponding message. If the processing succeeds, the job becomes `DONE` and the extracted metadata are persisted. If the processing fails, the job is either retried or, after the retry budget is exhausted, marked `FAILED`. These terminal states are terminal only for the job record; they do not stop the service itself.

Figure 9 captures the internal control flow of the worker service. The worker is intentionally long-running and therefore does not have a final lifecycle state in the usual sense: after handling one message, it returns to its idle listening state and waits for the next one. This diagram models a single message-processing cycle, starting from message reception, moving through parsing and processing, and then branching either to successful completion or to retry scheduling. On transient failures, the worker republishes the message with an incremented attempt counter and resets the job to `PENDING`; once the retry limit is reached, the worker records the final failure and stops retrying that job.

3.6 Data and Consistency Issues

The system stores both **persistent business data** and **ephemeral coordination data**. Persistent business data includes user accounts, authentication-related information, job metadata, job ownership, job state transitions, error messages, and extracted audio results. Binary audio files are not stored in the relational database: they are

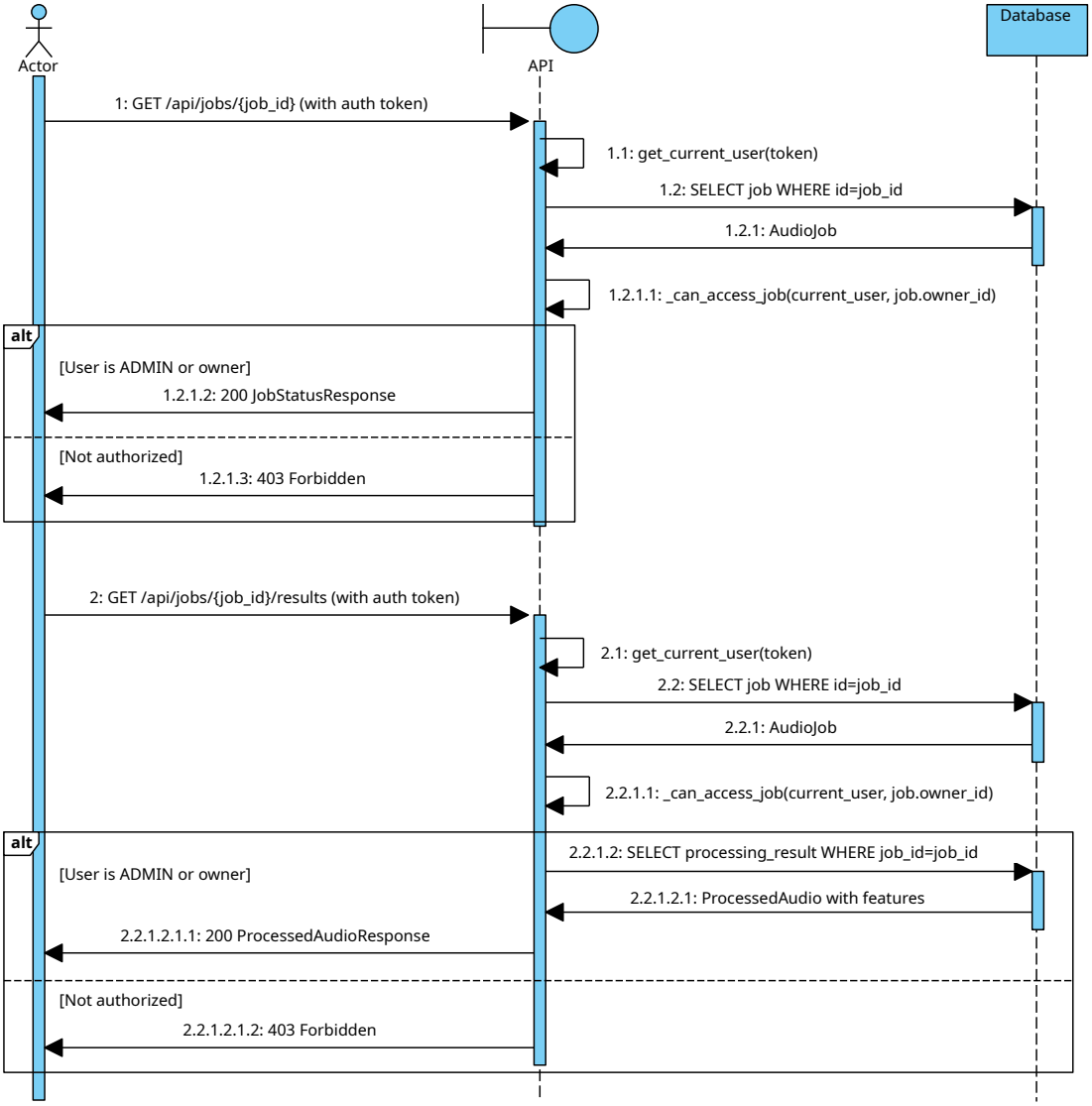

Figure 7: Sequence for polling job status and retrieving processing results once completion is reached.

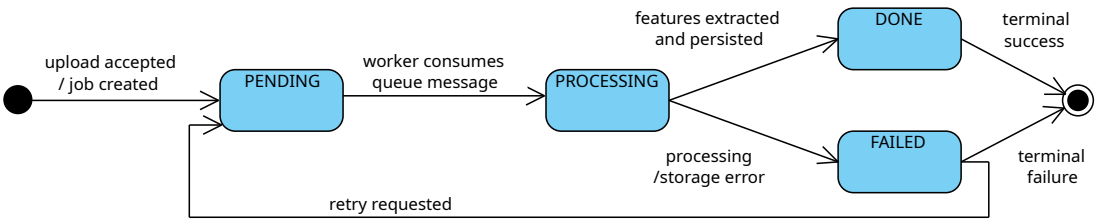

Figure 8: State diagram for the `AudioJob` lifecycle, from creation to completion or failure.

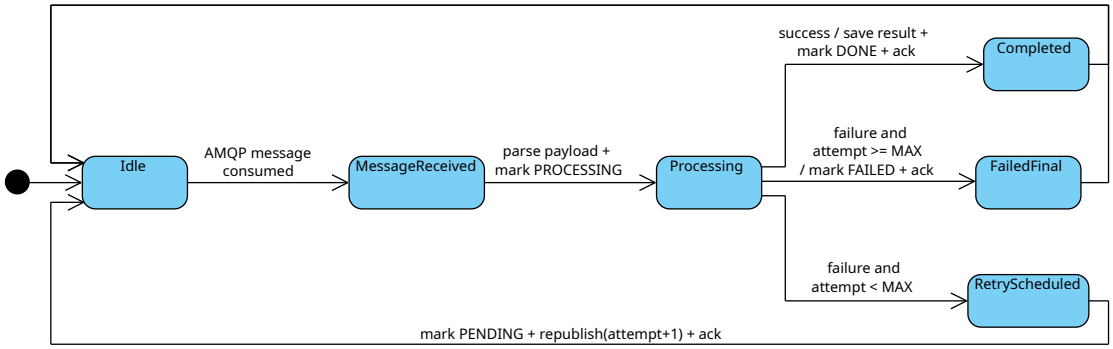

Figure 9: State diagram for the worker execution loop, including bounded retry handling.

persisted separately in MinIO as objects, while PostgreSQL keeps only the references needed to reconstruct the processing flow. This separation keeps the relational model compact and makes large file handling independent from metadata queries.

Persistent data is modeled as a relational schema because the main entities have clear identifiers, strong relationships, and structured lifecycle transitions. PostgreSQL is therefore the source of truth for users, jobs, and processing outcomes (as illustrated in Figure 10), while MinIO is the source of truth for uploaded audio content. The message broker is intentionally not used as the system of record: RabbitMQ carries work commands, but the durable state is always reconstructed from the database and object storage.

Several components perform queries on the database, but they do so with different responsibilities. The API service writes user and job data during registration, authentication, and upload handling, then reads jobs and results when serving status or result requests. The worker service updates job state during execution and writes processing results after a successful extraction attempt. Because the worker and API may access the same records concurrently, the design favors small, explicit updates rather than long transactions spanning multiple components.

Concurrent access is mostly read-heavy, with writes concentrated on job creation, status transitions, and result persistence. The main contention point is the `jobs` table, since the API initializes jobs and the worker advances their state. This is acceptable because each job is updated by a small number of transitions and the transitions are monotonic in normal execution. The `processing_results` table is updated with an upsert pattern, which keeps retries idempotent and avoids duplicate result rows when a message is reprocessed.

Some data must be shared between components, but only through durable storage and message payloads. The shared identifiers are the job ID, owner ID, storage object key, current state, timestamps, and extracted features. AMQP messages carry the job ID, object key, and retry attempt counter so the worker can continue execution without embedding business state in the queue itself.

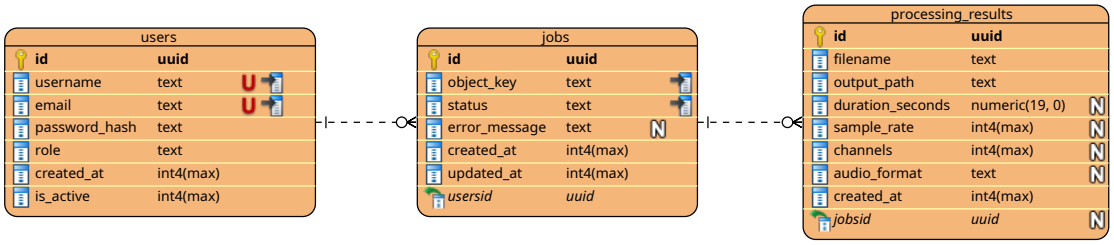

Figure 10: PostgreSQL schema entities and relationships.

3.7 Fault-Tolerance

Fault tolerance in this system is based on **durable state**, **asynchronous retries**, and **component isolation**. There is no separate replication layer for the application logic itself; instead, resilience comes from storing business data in **durable backends** and keeping the message broker as a **coordination layer** rather than as the source of truth. PostgreSQL stores job metadata, state transitions, and processing outcomes, while MinIO stores the uploaded audio objects. Because the persistent state is externalized, the API and worker services can restart independently without losing the information needed to resume processing.

Retry handling is applied to **transient failures** that can occur during processing or storage access. If a worker cannot complete a job because of a temporary issue, the message can be retried and the job remains recoverable until the retry budget is exhausted. This keeps the failure policy explicit and prevents the queue from silently discarding unfinished work.

Error handling is split across the components that can detect failures earliest. The API rejects invalid uploads and authentication failures immediately, so only valid jobs enter the system. The worker records processing failures in the job state, making them visible through the status and result endpoints and allowing operators to inspect persistent diagnostics. In this way, faults are converted into **observable job states** rather than hidden internal errors.

3.8 Availability

The system does not rely on a dedicated **caching** layer. This choice is deliberate: the most frequently accessed data are job metadata, processing state, and final results, all of which are already stored in durable backends and are small enough to retrieve directly. Avoiding cache invalidation logic keeps the system simpler and reduces the risk of serving stale job information.

Availability is instead improved through **load balancing** and **horizontal replication of stateless services**. In Kubernetes, the API can run with multiple replicas behind a Service, so incoming HTTP traffic is distributed across available instances. Worker replicas also scale horizontally and consume from the same RabbitMQ queue, which makes processing capacity more resilient to spikes in demand or temporary pod

restarts.

In case of **network partitioning**, the system degrades in a controlled way rather than hiding partial failures. If the API cannot reach the broker or the storage backends, uploads fail fast and return an error instead of accepting jobs that cannot be completed. If workers lose connectivity while processing, jobs remain persisted and can be retried once connectivity is restored. This behavior favors consistency and recoverability over masking broken communication paths, which is appropriate for an asynchronous processing pipeline.

3.9 Observability

Observability is treated as a first-class design concern because the pipeline is asynchronous and spans multiple loosely coupled components. The goal is not only to detect failures, but also to understand how requests move through the system, where work accumulates, and which component becomes a bottleneck under load. For this reason, the observability solution is built around **metric exposure**, **centralized scraping**, and **dashboard-based visualization**.

Each runtime component exposes Prometheus-compatible metrics that describe its own behavior. The API reports request volume and latency, the worker reports processing activity and failure counts, and the queue layer exposes backlog-related signals that indicate whether processing capacity is keeping up with demand. These metrics are aligned with the most relevant quality concerns of the project: responsiveness, throughput, reliability, and scaling behavior.

Prometheus acts as the collection layer by scraping the exposed metrics at regular intervals and storing them in a time-series format. Grafana then consumes those metrics to provide a consolidated view of the system, making it easier to inspect trends over time and correlate events such as upload bursts, queue growth, and worker slowdowns. This separation between metric production and visualization keeps the application code lightweight while still giving a clear operational picture.

From a design perspective, observability also supports validation and debugging. When a job remains pending for too long or a worker begins failing repeatedly, the metrics make the problem visible before it becomes a silent functional error. In that sense, the monitoring stack complements fault tolerance: fault tolerance preserves correctness under failure, while observability makes the failure itself measurable and actionable.

3.10 Security

The system implements **stateless, token-based authentication** and **role-based access control** to protect API endpoints and ensure that users can only access their own resources.

Authentication

Authentication is realized through **JWT (JSON Web Tokens)** issued on successful login. When a user submits valid credentials (username and password), the API verifies

them against the stored password hash and issues a JWT token. The token is signed using the HS256 algorithm and contains the user ID as the subject and an expiration time. The client includes the token in the `Authorization` header as a bearer token for all subsequent requests. Each protected endpoint applies a dependency that extracts and validates the token, extracting the user ID and loading the corresponding user record from the database. If the token is missing, malformed, or expired, the request is rejected with a 401 Unauthorized response. This approach keeps authentication stateless and allows API instances to verify tokens without session storage or coordination.

Authorization

The system uses **role-based access control (RBAC)** with three defined roles: `USER`, `ADMIN`, and `WORKER`. Standard users are granted the `USER` role on registration and can submit audio files, retrieve their own job status, and view their processing results. Job ownership is enforced at the application layer: queries filter results by the authenticated user's ID, so users can only see and interact with jobs they created. Administrative operations (user management, system configuration) are restricted to instances with the `ADMIN` role. The `WORKER` role is reserved for internal service-to-service communication in scenarios requiring explicit worker identification; it is not assigned to regular users. Authorization checks are implemented as FastAPI dependency injection, which transparently enforces access rules at the route level.

Cryptographic Schemas

The system employs two complementary cryptographic mechanisms:

- **Password hashing**: User passwords are hashed using bcrypt before storage in PostgreSQL. This ensures that even if the database is compromised, passwords remain protected. Verification happens through bcrypt comparison during login, not by storing plaintext or reversible encryption.
- **Token signing**: JWT tokens are signed with a shared secret key using HS256 (HMAC SHA-256). This cryptographic signature allows the API to verify that a token was genuinely issued by the system and has not been tampered with. The secret key is stored as an environment variable and is not hard-coded.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

> If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?

  - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
  - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
  - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?
  - e.g. OAuth, JWT, etc.
- how should components be *authorized*?
  - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework* / *technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components ***unit*-test**ed?
- how was communication, interaction, and/or integration among components tested?
- how to ***end-to-end*-test** the system?
  - e.g. production vs. test environment
- for each test specify:
  - rationale of individual tests
  - how were the test automated
  - how to run them
  - which requirement they are testing, if any

  recall that *deployment* **automation** is commonly used to *test* the system in *production-like* environment

  recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
  - what did you test?
  - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
  - provide instructions
  - provide expected outcomes
- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scrips (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
  - provide instructions
  - provide expected outcomes
  - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Discuss possible future works, improvements, extensions, optimizations, etc.

Also discuss here any unimplemented feature, and how you would have implemented it.

References

[1] D. Adams. *The Hitchhiker's Guide to the Galaxy.* San Val, 1995.